

elttam/publications: writeups/home-assistant/signed-url-param-tampering-advisory.md

elttam/publications: writeups/home-assistant/signed-url-param-tampering-advisory.md - elttam, GitHub.

- Published: date not stated
- Original: <<https://github.com/elttam/publications/blob/master/writeups/home-assistant/signed-url-param-tampering-advisory.md>>
- Preserved from: <https://github.com/elttam/publications/blob/master/writeups/home-assistant/signed-url-param-tampering-advisory.md> (github-api) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Repository reading copy. Created from [signed-url-param-tampering-advisory.md](#).

signed-url-param-tampering-advisory.md

elttam/publications at master , path writeups/home-assistant/signed-url-param-tampering-advisory.md .

Introduction

This write-up describes a vulnerability found in [Home Assistant](#), a popular open source home automation software. The vulnerability was found to affect versions before 2023.3.6 where a patch is introduced.

Overview

In the [HTTP integration](#), requests can be authenticated with either a bearer token in the Authorization header, or with a signature (for GET requests) in the query string. A vulnerability was found in the signature validation function which allowed an attacker to tamper with a request's query parameters to reuse a valid signature that would authenticate a request under a different set of query parameters

Timeline

- **22/03/2023** - We discover the vulnerability
- **23/03/2023** - Vulnerability report sent to Home Assistant developers
- **23/03/2023** - Home Assistant replies, noting that a [patch](#) was made
- **23/03/2023** - Home Assistant 2023.3.6 is released, containing the patch which fixes the issue
- **10/05/2023** - Public release of this advisory

Description

The implementation of the auth middleware is [here](#):

```
@middleware
async def auth_middleware(
    request: Request, handler: Callable[[Request], Awaitable[StreamResponse]]
) -> StreamResponse:
    """Authenticate as middleware."""
    authenticated = False

    if hdrs.AUTHORIZATION in request.headers and await async_validate_auth_header(
        request
    ):
        authenticated = True
        auth_type = "bearer token"

    # We first start with a string check to avoid parsing query params
    # for every request.
    elif (
        request.method == "GET"
        and SIGN_QUERY_PARAM in request.query_string
        and await async_validate_signed_request(request)
    ):
        authenticated = True
        auth_type = "signed request"

    if authenticated:
        _LOGGER.debug(
            "Authenticated %s for %s using %s",
            request.remote,
            request.path,
            auth_type,
        )

    request[KEY_AUTHENTICATED] = authenticated
    return await handler(request)
```

As seen in the `elif` block, for GET requests, it is possible to authenticate the request if a signature is present in the query string. The signature validation is performed in the `async_validate_signed_request` method:

```
async def async_validate_signed_request(request: Request) -> bool:
    """Validate a signed request."""
    if (secret := hass.data.get(DATA_SIGN_SECRET)) is None:
        return False

    if (signature := request.query.get(SIGN_QUERY_PARAM)) is None:
        return False

    try:
        claims = jwt.decode(
            signature, secret, algorithms=["HS256"], options={"verify_iss": False}
        )
    except jwt.InvalidTokenError:
        return False

    if claims["path"] != request.path:
        return False

    params = dict(sorted(request.query.items()))
    del params[SIGN_QUERY_PARAM]
    for param in SAFE_QUERY_PARAMS:
        params.pop(param, None)
    if claims["params"] != params:
        return False

    refresh_token = await hass.auth.async_get_refresh_token(claims["iss"])

    if refresh_token is None:
        return False

    request[KEY_HASS_USER] = refresh_token.user
    request[KEY_HASS_REFRESH_TOKEN_ID] = refresh_token.id
    return True
```

A vulnerability exists here in the validation of the query params. The `claims["params"]` object provided in the signature body is checked against `dict(sorted(request.query.items()))` which is derived from the current request's query parameters.

Here, `request` is an `aiohttp Request` object, and `request.query` is a `MultiDictProxy` containing the query parameters. The `.items` method returns a list of tuples containing

the `(key, value)` pairs corresponding to each query parameter. Importantly, `aiohttp` supports multiple query parameters with the same name through this, so providing a query string such as `x=1&x=2` will result in the `request.query.items()` being `[('x', '1'), ('x', '2')]`. Because of the way this is handled in the signature validation, when multiple values are specified, some of them will be ignored. Specifically, when it is used to construct the `dict` object, the "largest" (by string comparison) value will be taken and the "smaller" values will be ignored.

However, when using query parameters in request handlers via the `request.query.get(param_key)` method, the *first* parameter that appears in the query string is taken. This means the parameter being validated and the parameter actually used by a handler may differ.

As an example, suppose some functionality signs the path `/test?id=7` and we obtain the signature for this path. In this case, `claims["params"]` will be `{ "id": 7 }`. If we attempt to access the path `/test?id=6&id=7` using the same signature, it will be authorised since `dict(sorted(request.query.items())) == { "id": 7 }`, but the handler for that path will use `id=6` (if the `id` is obtained via a call to `request.query.get("id")`).

Proof of Concept

The following test case (in `tests/components/http/test_auth.py`) can be run to verify the issue:

```

@pytest.mark.parametrize(
    ("base_url", "test_url", "chosen_param_value"),
    [
        ("/test?id=7", "/test?id=6&id=7", "6"),
    ],
)
async def test_auth_access_signed_path_with_multiple_query_param_tamper(
    hass: HomeAssistant,
    app,
    aiohttp_client: ClientSessionGenerator,
    hass_access_token: str,
    base_url: str,
    test_url: str,
    chosen_param_value: str,
) -> None:
    """Test access with signed url and query params that have been tampered with using
    repeated parameters."""
    async def mock_handler(request):
        """Return if request was authenticated."""
        if not request[KEY_AUTHENTICATED]:
            raise HTTPUnauthorized

        user = request.get("hass_user")
        user_id = user.id if user else None

        assert request.query.get('id') == chosen_param_value
        return web.json_response(data={"user_id": user_id})

    app.router.add_get("/test", mock_handler)
    await async_setup_auth(hass, app)
    client = await aiohttp_client(app)

    refresh_token = await hass.auth.async_validate_access_token(hass_access_token)

    signed_path = async_sign_path(
        hass, base_url, timedelta(seconds=5), refresh_token_id=refresh_token.id
    )
    url = yarl.URL(signed_path)
    token = url.query.get(SIGN_QUERY_PARAM)

    req = await client.get(f"{test_url}&{SIGN_QUERY_PARAM}={token}")
    assert req.status == HTTPStatus.UNAUTHORIZED

```

With the implementation of the signature validation as of Home Assistant Core 2023.3.5, this test will fail as the final assertion will not hold due to the request being improperly authorised.

Impact

There were no cases identified which used query parameters in signed urls. Furthermore, to exploit this issue, an attacker would need to obtain a signed URL in the first place, which may significantly decrease the likelihood.

This may potentially be an issue if in the future components or integrations are implemented which depend on this feature and use query parameters for sensitive fields (such as IDs or file paths).

Discovered

- March 2023, Joseph Surin, elttam

Archived from <https://github.com/elttam/publications/blob/master/writeups/home-assistant/signed-url-param-tampering-advisory.md>. Rendered offline from the local Markdown copy.